

Stack (DFS)

Mijn code begint met de startpositie op een stack te plaatsen. In de while-loop popt steeds de bovenste positie af. Als die positie het eindpositie is, stopt het. Anders dan kijkt het naar alle andere kanten via `maze.moves` (omhoog, omlaag, links en rechts). Elke positie die valid is, nog niet bezocht is, en een vrije cel 0 of eindpunt 2 is, wordt op de stack gegooit. Op hetzelfde moment slaat het op in de parent-dictionary vanuit welke cel die daarna bereikt werd.

Omdat het Stack is (LIFO), gaat het altijd verder op de laatste richting die hij bezocht, zo diep mogelijk, voordat hij terugkeert. Dat is de depth first search. Als het eindpunt gevonden is, loopt `ReconstructPath` terug via de parent-dictionary van eindpunt naar beginpunt, draait die om, en zet alles in de queue.

A*

De A* werkt anders: in plaats van een stack gebruikt hij een `SortedSet<Node>` die altijd gesorteerd is op F-waarde (laagste eerst). $F = G + H$, waarbij G het aantal stappen vanaf de start is, en H de Manhattan-afstand tot het eindpunt. Bij elke stap pakt hij de node met de laagste F (`openSet.Min`). Als dat het eindpunt is, stopt het. Anders gaat die naar eentje daarnaast. Voor elke daarnaast berekent hij een nieuwe G (`current G + 1`). Is die beter dan wat hij eerder had voor die cel, dan werkt hij de parent bij en voegt degene daarnaast opnieuw toe aan de open set met de nieuwe scores.

De `closedSet` zorgt ervoor dat al volledig verwerkte cellen niet opnieuw worden bekeken. De `NodeComparer` sorteert op F, dan G, dan invoegvolgorde, zodat bij delijke scores dezelfde keuze gemaakt wordt. (Net als bij DFS herstelt `ReconstructPath` aan het einde het pad via de parent-dictionary.)

Recursie

Voor dit onderdeel heb ik ervoor gekozen om zowel de maze generator als de path finder recursief te implementeren. Hierdoor gebruiken beide algoritmes dezelfde denkwijze en sluiten ze inhoudelijk goed op elkaar aan.

Dat gaf twee voordelen:

- De logica blijft relatief compact en overzichtelijk.
- De visualisatie werd veel duidelijker, omdat je letterlijk ziet hoe het algoritme stap voor stap de diepte in gaat.

Door dezelfde recursieve aanpak te gebruiken, wordt het bovendien eenvoudiger om de werking van beide algoritmes met elkaar te vergelijken.

Hoe mijn recursive path finder werkt:

De path finder start bij het beginpunt van de maze en onderzoekt vervolgens recursief alle geldige buurcellen. Zodra het eindpunt wordt bereikt, stopt het algoritme direct met zoeken.

Een belangrijk onderdeel van mijn implementatie is het bijhouden van parent-relaties. Hiermee wordt opgeslagen vanaf welke positie een cel is bereikt. Wanneer het eindpunt gevonden is, kan ik daardoor achteraf het volledige correcte pad reconstrueren.

Voorbeeld van de stopconditie:

```
if (current[0] == maze.End[0] && current[1] == maze.End[1])
{
    found = true;
    endPosition = current;
    return;
}
```

Tijdens het zoeken doe ik ook markeringen voor visualisatie:

- 4 voor verkend;
- 5 voor fout spoor (backtracking).

Daardoor zie je niet alleen de finale route, maar ook alle mislukte vertakkingen die het algoritme eerst probeerde.

Voorbeeld van parent-registratie en verkenning:

```

if (cellValue == 0 || cellValue == 2)
{
    parent[newKey] = current;
    visitedPositions.Enqueue(newPos);

    if (maze.MazeArray[newRow][newCol] != 2)
        maze.MazeArray[newRow][newCol] = 4;

    Recurse(newPos);

    if (!found)
    {
        maze.MazeArray[newRow][newCol] = 5;
        visitedPositions.Enqueue(newPos);
    }
}

```

Deze aanpak zorgt ervoor dat het volledige zoekproces zichtbaar blijft en maakt het eenvoudiger om de werking van het algoritme te analyseren.

Dijkstra pathfinding

De code begint met een datastructuur op te zetten. Dist= is de afstand van de start, parent= wijst aan welke cel ligt op het pad, dit is nodig om de kortste route te vinden, visited= cellen die al bezocht zijn en pq= is de priority queue, deze geeft eerst de cel met kortste afstand van start terug.

Startcel dist=0, wordt in queue gezet. Dequeue de cel met laagste afstand van de start. Als het in visited zit wordt het overgeslagen. Ongemarkeerde cel wordt gelabeld met nummer 4 en check of de cel 2 is, zo ja stop het proces.

Check cellen de boven, beneden en naast of het geen muur (-1) is of een cel die al visited is. dist wordt geüpdatet als de afstand korter is van de huidige, de parent opslaan en enqueue.

Wanneer je het einde hebt gevonden volg je de parent[] en path.reverse is om de route van start te hebben.

Maze generator

Het begint met een grid vol muren (-1) en carft gangen open. Hij gebruikt Dijkstra met willekeurige randgewichten zodat elke keer een ander doolhof ontstaat. Bij de generator krijgt elke rand een willekeurig getal van 1 tot 999. Hierdoor kiest Dijkstra een willekeurige volgorde van cellen om te carven, hiermee krijgt je willekeurig patronen voor de maze.

Cellen staan altijd op oneven rij en kolom: (1,1), (1,3), (3,1) etc. De even posities ertussen zijn de muren. Door 2 stappen te zetten spring je van cel naar cel, en het midpunt is de muur die je opent. De generator gebruikt een zelfgeschreven priority queue op basis van een binairesearchtree. Kleinere prioriteiten gaan naar links, grotere naar rechts. Het minimum is altijd de meest linkse knoop.

Wanneer het klaar is, loopt hij vanuit de start door alle gangen en zoekt de cel die het verste weg ligt. Dat wordt het eindpunt.

Hoe mijn recursive maze generator werkt:

Voor de generator gebruik ik depth-first carving met sprongen van 2 cellen. Ik begin op (1,1), kies een willekeurige richting, breek de tussenmuur open en gaat de recursie dan verder.

In plaats van één cel tegelijk te bewegen, wordt steeds twee cellen vooruitgekeken. Hierdoor blijven muren behouden tussen de paden. Wanneer een nieuwe cel wordt gekozen, wordt de tussenliggende muur verwijderd en gaat het algoritme recursief verder vanuit de nieuwe positie. Deze aanpak zorgt voor een natuurlijke doolhof.

Voorbeeld van de carve-stap:

```
grid[row + dRow / 2][col + dCol / 2] = 0;  
  
CarvePassagesRecursive(grid, visited, nextRow, nextCol);
```

Na het genereren van de maze wordt het eindpunt niet willekeurig geplaatst. In plaats daarvan gebruik ik een **Breadth-First Search (BFS)** om vanaf het startpunt het verste bereikbare punt te vinden.

Hierdoor wordt de afstand tussen start en eindpunt meestal groter, wat resulteert in een interessantere en uitdagendere maze.

Voorbeeld van de BFS-uitbreiding:

```
distance[nextRow][nextCol] = currentDistance + 1;  
  
queue.Enqueue(new int[] { nextRow, nextCol });
```

Deze methode levert doorgaans een langere oplossingsroute op dan wanneer het eindpunt willekeurig wordt gekozen.

Wat ik sterk vind aan mijn eigen aanpak:

- ❖ De generator en solver passen inhoudelijk bij elkaar door dezelfde recursieve stijl.
- ❖ De visualisatie laat echt het denkproces zien, niet alleen het eindantwoord.
- ❖ Door het verste eindpunt te kiezen wordt de maze merkbaar uitdagender.
- ❖ De structuur laat toe om later eerlijk te vergelijken met Dijkstra, A* en stack-based oplossingen.

Wat ik nog kan verbeteren:

- ❖ [Een iteratieve fallback met een expliciete stack zou dit robuuster maken.
- ❖ Ik kan extra metrics toevoegen, zoals bezochte knopen, totale tijd en pad lengte, zodat de vergelijking tussen algoritmes objectiever wordt.

Conclusie:

Door zowel de maze generator als de path finder recursief op te bouwen, heb ik een consistente en overzichtelijke oplossing ontwikkeld. De combinatie van recursie, visualisatie en het gebruik van BFS voor het bepalen van een geschikt eindpunt zorgt voor een goed inzicht in de werking van de algoritmes. Daarnaast biedt de huidige structuur een sterke basis voor toekomstige uitbreidingen en vergelijkingen met andere zoekalgoritmes.